

第十二章 深度确定性策略梯度

深度确定性策略梯度 (DDPG)

全称 Deep Deterministic Policy Gradient，其实就是第九章里面路径衍生策略梯度的具体实现。

详细内容见第九章该部分笔记。

这个算法有一些问题：最主要的就是和 DQN 一样，只要一开始估计了一个异常偏高的 Q 值，就会直接导致利用这一异常高值，循环往复导致**严重高估**。

下面是对该算法的一些补充和改进。

双延迟深度确定性策略梯度 (TD3)

全称 Twin Delayed Deep Deterministic Policy Gradient，是在 DDPG 的基础上应用了 3 个技巧升级而来的，具体如下：

1. 截断的双 Q 学习 (Clipped Double Q -Learning)

和之前第七章从 DQN 到 DDQN 的思想有点类似：既然“出现异常高估就选择该高估值”不可取，就尽量破坏这种必然关系。

DDQN 采用的是评价和选择网络分离，由于两个网络在同一动作处产生异常高估的概率很小，很大程度上避免了异常高估。

而此处的截断的双 Q 学习采用了类似的思路：建立两个不同的 Critic，预测得到两个 Q 值，**取其中较小值**。理由是一样的：两个网络的高估点匹配上的概率要小得多。

(注意：这样做其实会导致理论上的低估，但这是可以接受的。本质上这里就是**主动牺牲一些无偏性，换取对严重高估的抑制**。)

直观理解：**两个独立评委，取更保守的得分**。

2. 延迟的策略更新 (Delayed Policy Updates)

我们看为什么会产生误差以及高估：本质上就是 Critic 网络没学到位，偏差非常大，Actor 去学一个和真实 Q 差十万八千里的目标，自然很难学好，表现为训练效果差，难以收敛，方差大。既然如此，我们可以**让 Critic 多更新而 Actor 少更新**，让 Critic 拥有更多时间拟合 TD 目标，然后 Actor 再根据相对可靠的 Critic 进行更新。

直观理解：Critic 负责频繁修正评分标准，Actor 不用每次评分刚变化一点就立刻改动作，而是等评分标准稳定一些后再调整策略。

3. 目标平滑策略 (Target Policy Smoothing)

一般来说，实际操作中如果有连续动作，随着动作的细微变化，奖励变化不应当过于剧烈（例如角度选 14.99° 和 15.01° 其实差不太多），而出现误差的 Critic 产生的 Q 会呈现出在某一点突然“冒尖”，也就是虚假尖峰。

那既然这样，我们可以选择不采用“冒尖”的动作，而是采用其附近的、可能没有那么高估的动作，符号化语言即：

$$a_{TD3}(s') = \text{clip}(\mu_{\bar{\theta}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{low}}, a_{\text{high}}), \quad \epsilon \sim \mathcal{N}(0, \sigma).$$

解释一下这个 clip 函数：

$$\text{clip}(f_x, a, b) = \begin{cases} a, & f_x \leq a, \\ f_x, & a < f_x < b, \\ b, & f_x \geq b. \end{cases}$$

直观理解：比如角度选择，从 49.5–50.5 真实的 Q 应该差不多，但是 Critic 给出了在 50 左右突然有一个尖峰。此时我们不选 50，而是通过噪声给选择加上一定的偏移量，例如可能选 49.8、50.3 之类的，来避免出现过度高估。

值得注意的是，这三个技巧属于**提高工程稳定性**的设计，并不意味着算法在任何环境、任何超参数下都必然收敛或必然优于其他算法。

TD3 一轮完整的训练流程

结合以上三点优化，我们得到了 TD3 的一轮完整训练流程：

1. Actor 与环境交互输出确定性动作，加入**探索噪声**：

$$a_{\text{env}} = \text{clip}(\mu_{\theta}(s) + \epsilon_{\text{explore}}, a_{\text{low}}, a_{\text{high}})$$

然后执行动作，得到一条经验 (s, a, r, s', d) ，放入经验回放池。

2. 从经验回放池中随机采取一批样本，batch 大小为 B ：

$$\{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^B$$

3. 计算带平滑的目标动作：

$$a'_{TD3} = \text{clip}(\mu_{\bar{\theta}}(s') + \text{截断噪声}, a_{\text{low}}, a_{\text{high}})$$

※这里使用了上面的第三条：目标平滑策略。

4. 计算保守的 TD 目标：

$$y_i = r_i + \gamma(1 - d_i) \min [Q_{\phi_{1,\text{targ}}}(s'_i, a'_{\text{TD3}}), Q_{\phi_{2,\text{targ}}}(s'_i, a'_{\text{TD3}})]$$

※这里使用了上面的第一条：截断的双 Q 学习。

5. 分别更新两个 Critic：独立构造损失然后最小化：

$$L_{\text{critic},1} = \frac{1}{B} \sum_i [Q_{\phi_1}(s_i, a_i) - y_i]^2,$$
$$L_{\text{critic},2} = \frac{1}{B} \sum_i [Q_{\phi_2}(s_i, a_i) - y_i]^2.$$

6. 延迟更新 Actor：只有每隔若干次 Critic 更新后，才更新 Actor：

$$L_{\text{actor}} = -\frac{1}{B} \sum_i Q_{\phi_1}(s_i, \mu_{\theta}(s_i))$$

(注意：虽然前面有两个 Critic 网络，但是这里通常使用其中一个在线 Critic 给 Actor 提供梯度进行更新。)

※这里使用了上面的第二条：延迟的策略更新。

7. 和 DDPG 一样，目标网络隔一段时间更新到在线网络，循环往复。

DDPG 与 TD3 对比

方面	DDPG	TD3
Critic 数量	1 个	2 个
TD 目标	使用一个目标 Critic	使用两个目标 Critic 的较小值
Actor 更新	较频繁	延迟更新
目标动作	直接由目标 Actor 输出	加入截断噪声后再计算
对 Q 高估的处理	没有专门抑制	双 Critic 取最小值
对 Q 尖峰的处理	没有专门处理	目标策略平滑
训练稳定性	对初始化和超参数较敏感	通常更稳定

总结：TD3 在 DDPG 的基础上，让 Critic 变得更保守、更平滑，让 Actor 不要过早、过度地相信一个尚未学好的 Critic。